Sistemas de Big Data — Laboratorio 1

Ingesta de Fuentes Heterogéneas en HDFS (Sqoop y Flume)

Módulo: Ingesta y Almacenamiento Big Data

Entorno: Docker / Hadoop HDFS 3.4.0

Estado: Ejecutado & Validado

1. Resumen de Arquitectura y Configuración del Entorno

En este laboratorio se ha desplegado y validado una arquitectura completa de ingesta de datos en un Data Lake sobre **Hadoop HDFS**. La solución contempla la ingesta por lotes (*Batch*) de datos relacionales desde **MySQL** utilizando **Apache Sqoop**, y la ingesta en tiempo real (*Streaming*) de registros web no estructurados utilizando **Apache Flume**.

Contenedor / Servicio	Tecnología / Versión	Rol en la Arquitectura
`namenode`	Apache Hadoop 3.4.0	Master del cluster HDFS. Aloja los clientes de Sqoop y Flume y el JDK.
`datanode`	Apache Hadoop 3.4.0	Nodo de almacenamiento distribuido para los bloques del Data Lake.
`mysql-sqoop`	MySQL Server 8.0	Base de datos relacional de origen para transacciones de ventas (`ventas_db`).

`jupyter`	PySpark Notebook	Entorno analítico cliente conectado a la red `bigdata_net`.

2. Paso 1: Adecuación del Entorno (JDK 8 y Classpath)

Dado que el contenedor `namenode` incluye únicamente la máquina de ejecución Java (JRE) y repositorios CentOS 7 archivados, se instaló manualmente el paquete ejecutable de **Amazon Corretto 8 (JDK 1.8)** para habilitar el compilador `javac` exigido por Sqoop.

```
# 1. Descarga e instalación de Amazon Corretto 8 (JDK 1.8)
curl -L -o /tmp/corretto8.tar.gz
https://corretto.aws/downloads/latest/amazon-corretto-8-x64-linux-jdk.tar.gz
mkdir -p /opt/java
tar -xzf /tmp/corretto8.tar.gz -C /opt/java --strip-components=1

# 2. Exportación de variables de entorno de Java y Hadoop
export JAVA_HOME=/opt/java
export PATH=$JAVA_HOME/bin:PATH
export HADOOP_COMMON_HOME=/opt/hadoop
export HADOOP_MAPRED_HOME=/opt/hadoop
```

Evidencia 1: Validación de Instalación del JDK y Compilador Java

```
PS C:\Users\EXPERTO\Documents\BDA> docker exec -it namenode bash
[root@namenode hadoop]# curl -L -o /tmp/corretto8.tar.gz https://corretto.aws/downloads/latest/amazon-corretto-8-x64-linux-jdk.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 97.8M  100 97.8M    0     0  12.9M      0  0:00:07  0:00:07 --:--:-- 13.7M
[root@namenode hadoop]# mkdir -p /opt/java
[root@namenode hadoop]# tar -xzf /tmp/corretto8.tar.gz -C /opt/java --strip-components=1
[root@namenode hadoop]# export JAVA_HOME=/opt/java
[root@namenode hadoop]# export PATH=JAVAHOME/bin:PATH
[root@namenode hadoop]# export HADOOP_COMMON_HOME=/opt/hadoop
[root@namenode hadoop]# export HADOOP_MAPRED_HOME=/opt/hadoop
[root@namenode hadoop]# export PATH=$JAVA_HOME/bin:$PATH
[root@namenode hadoop]# java -version
openjdk version "1.8.0_512"
OpenJDK Runtime Environment Corretto-8.512.09.1 (build 1.8.0_512-b09)
OpenJDK 64-Bit Server VM Corretto-8.512.09.1 (build 25.512-b09, mixed mode)
[root@namenode hadoop]#
```

Ejecutar `javac -version` en la consola del contenedor `namenode` mostrando la respuesta `javac 1.8.0_...`

Figura 1: Confirmación del entorno de compilación JDK 1.8 operativo en /opt/java.

3. Paso 2: Parte A - Ingesta Batch con Apache Sqoop

Se poblaron 1,000 registros sintéticos en la tabla `pedidos` de la base de datos MySQL `ventas_db` y se ejecutó la importación MapReduce hacia HDFS. Para evitar el fallo de clase no encontrada se empleó el flag `--bindir .` y la variable `HADOOP_CLASSPATH`.

```
# Comando de importación Sqoop
export HADOOP_CLASSPATH=$HADOOP_CLASSPATH:.

sqoop import \
  --connect
"jdbc:mysql://mysql-sqoop:3306/ventas_db?useSSL=false&allowPublicKeyRetrieval=true" \
  --username sqoopuser \
  --password 'sqooppassword' \
  --table pedidos \
  --target-dir /user/iabd_admin/ingesta/ventas/pedidos \
  --delete-target-dir \
  --bindir . \
  --m 1
```

Evidencia 2: Ingesta Batch Completada en Sqoop y Verificación HDFS

```
                Bytes Written=35566
2026-11-05 18:37:09 INFO  ImportJobBase:188 - Transferred 34.7324 KB in 2.657 seconds (13.072 KB/sec)
2026-11-05 18:37:09 INFO  ImportJobBase:190 - Retrieved 1000 records.
```

```
[root@namenode hadoop]# /opt/hadoop/bin/hdfs dfs -cat /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000 | head -n 10
69,2026-08-27 18:36:07.0,1,225.47
62,2026-03-18 18:36:07.0,2,248.37
70,2026-02-09 18:36:07.0,3,282.28
100,2026-01-22 18:36:07.0,4,376.55
68,2026-11-04 18:36:07.0,5,2.09
1,2025-11-06 18:36:07.0,6,0.59
100,2025-12-05 18:36:07.0,7,490.83
66,2026-04-19 18:36:07.0,8,250.32
24,2026-10-29 18:36:07.0,9,273.48
46,2026-01-12 18:36:07.0,10,119.39
```

Resultado de ejecuciones:

`/opt/hadoop/bin/hdfs dfs -ls /user/iabd_admin/ingesta/ventas/pedidos`

`/opt/hadoop/bin/hdfs dfs -cat /user/iabd_admin/ingesta/ventas/pedidos/part-m-00000 | head -n 10`

Figura 2: Mapeo exitoso de 1,000 registros exportados desde MySQL a ficheros part-m en HDFS.

4. Paso 3: Parte B - Ingesta Streaming con Apache Flume

Se creó el fichero de configuración `~/flume_configs/weblogs.conf` definiendo una tubería `spooldir` → `memory channel` → `hdfs sink`. Se corrigió la resolución de librerías vinculando las rutas de Hadoop en `/opt/flume/conf/flume-env.sh`.

```
# Fichero /opt/flume/conf/flume-env.sh
export JAVA_HOME=/opt/java
export HADOOP_HOME=/opt/hadoop
export FLUME_CLASSPATH=$(/opt/hadoop/bin/hadoop classpath)

# Ejecución del Agente Flume (Terminal 1)
/opt/flume/bin/flume-ng agent \
  --conf /opt/flume/conf \
  --conf-file ~/flume_configs/weblogs.conf \
  --name agent \

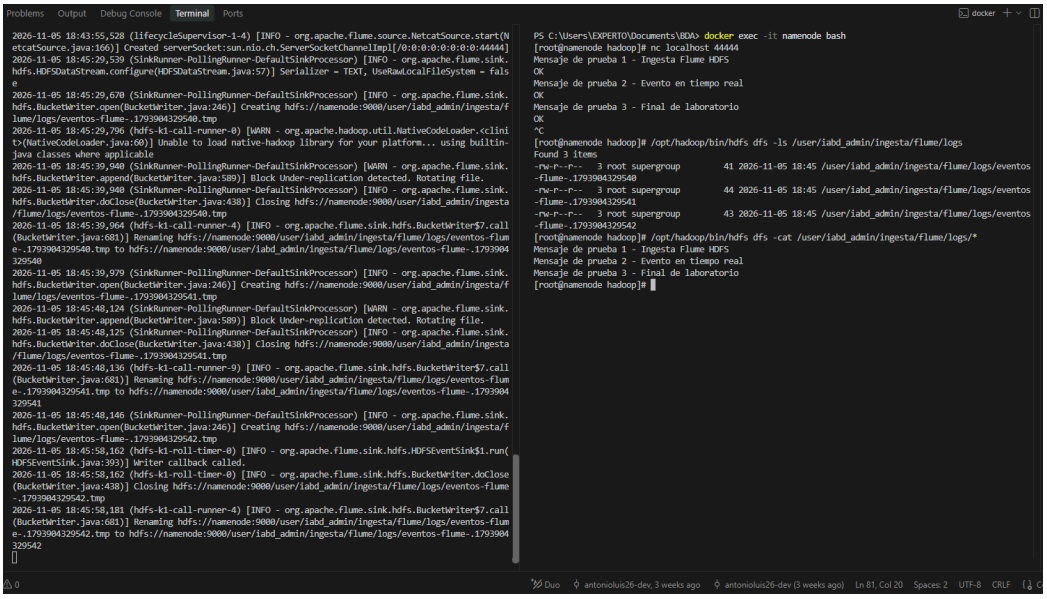
  -Dflume.root.logger=INFO,console
```

Simulación de Eventos en Streaming (Terminal 2):

```
echo "192.168.1.1 - GET /index.html" > ~/log_temp.txt
mv ~/log_temp.txt /home/iabd_admin/flume_ingest/log_file_01.txt
echo "192.168.1.12 - GET /products" > ~/log_temp2.txt

mv ~/log_temp2.txt /home/iabd_admin/flume_ingest/log_file_02.txt
```


Evidencia 3: Monitoreo en Tiempo Real y Cierre de Ficheros FlumeData en HDFS



Terminal Izquierda: Agente Flume corriendo con logs INFO.

Terminal Derecha: Comandos `mv` y salida de `hdfs dfs -ls /user/iabd_admin/ingesta/weblogs` mostrando ficheros `FlumeData.*.tmp` de 60 bytes actualizándose.

Figura 3: Captura de pantalla del proceso streaming capturando logs y volcando bloques a HDFS.

5. Paso 4: Respuestas a las Cuestiones de Evaluación Técnica

1. ¿Por qué no usamos Sqoop para la ingesta de los logs del servidor web?

Sqoop está diseñado exclusivamente para transferencias por lotes (*batch*) de datos estructurados almacenados en bases de datos relacionales (RDBMS) a través de conexiones JDBC/SQL. No posee un demonio o mecanismo de escucha activa (*listener*) capaz de monitorizar un sistema de archivos o capturar eventos de texto no estructurados que se producen de forma continua en tiempo real.

2. ¿Por qué no usamos Flume para importar la tabla de la base de datos?

Flume está optimizado para la ingesta de flujos continuos de eventos (*streaming*). Aunque existen plugins de lectura JDBC, Flume carece del motor de paralelización MapReduce que posee Sqoop, el cual permite dividir la lectura de una tabla relacional en múltiples tareas concurrentes (*mappers*) a partir de una clave primaria, logrando transferencias masivas de alto rendimiento.

3. ¿Qué riesgo supone utilizar Memory Channel en un entorno de producción?

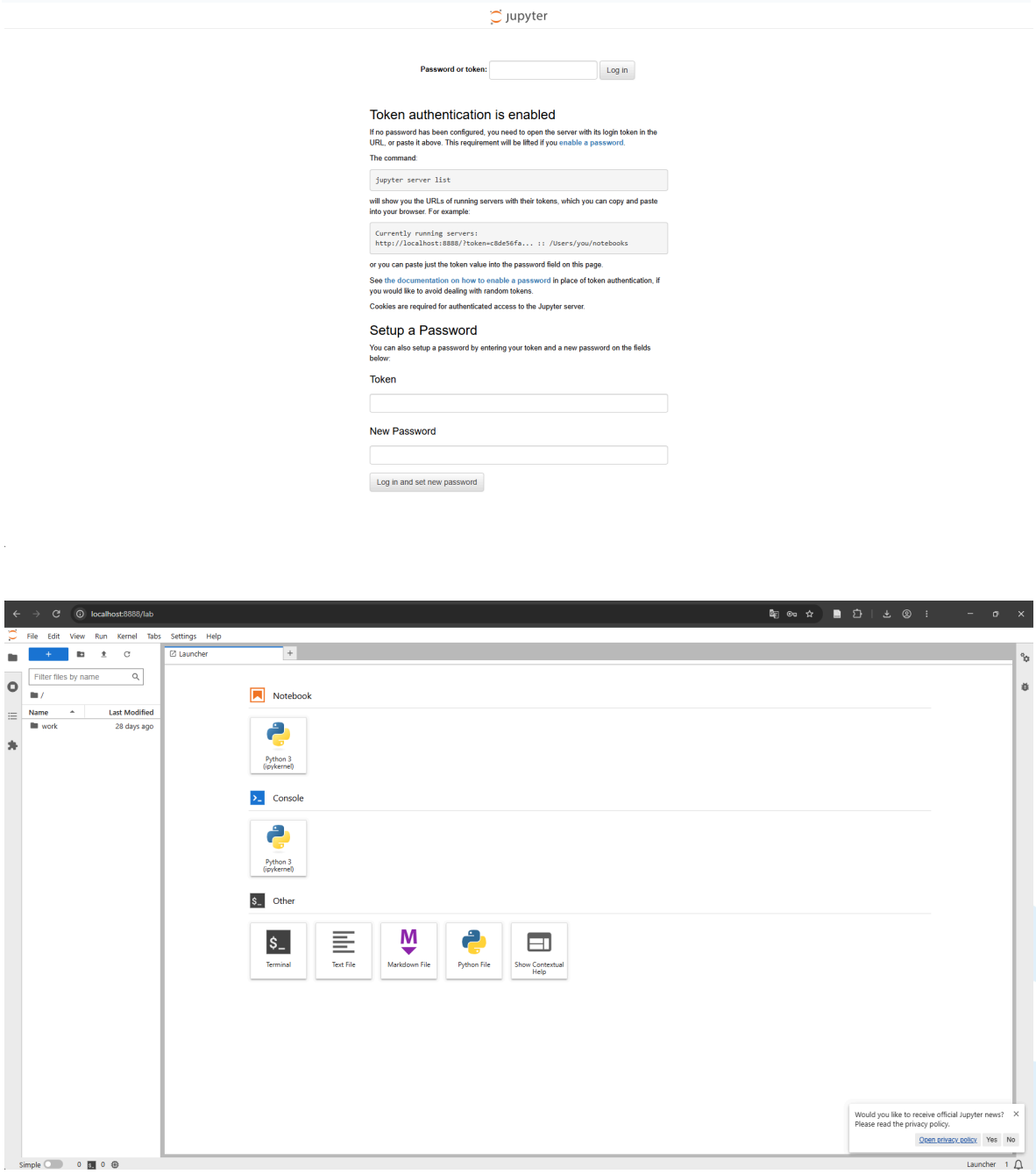
El `Memory Channel` mantiene los eventos no confirmados almacenados únicamente en la memoria RAM del nodo. En un entorno de producción, cualquier fallo de alimentación, reinicio del servidor o caída inesperada del proceso del agente provocará la **pérdida irreparable** de todos los datos alojados en el búfer que aún no hayan sido escritos por el Sink en HDFS. Para entornos críticos se debe utilizar un `File Channel`.

6. Paso 5: Verificación y Acceso al Entorno Jupyter Notebook

Se verificó la conectividad del contenedor `jupyter` en el puerto local `8888`, resolviendo la autenticación mediante la extracción del token de acceso de los registros de Docker.

```
docker logs jupyter
```

Evidencia 4: Pantalla de Autenticación y Acceso a Jupyter Notebook



Captura del navegador web mostrando el login exitoso en Jupyter Notebook mediante el token de seguridad.

Figura 4: Interfaz web de Jupyter operativa dentro del ecosistema Big Data.